

Month 1: Week 3

Git & Version Control

👨‍💻 What is Git and Why does it matter?

➔ Have you ever saved a file as 'final', then 'final_v2', then 'final_ACTUAL_final'?" That's manual version control, and it breaks the moment two people are working on the same thing.



Git solves this. It tracks every change to your code, who made it, and when. GitHub is where you store that code online so a team can collaborate on it.

- Git = the tool that runs on your machine
- GitHub = A website that hosts your repositories online, there are other websites like GitHub for the same purpose too

They are not the same thing. Git works without GitHub. GitHub is built on top of Git.

🧑‍💻 Git

Git is a version control system, a tool that helps developers track changes to their code over time.

Think of it like a time machine for your project, you can go back to previous versions, see who changed what, when, and why, and even work on new features without breaking the main codebase.

Imagine you're writing a book with your team. Git helps everyone:

- Keep their own copies
- Track edits
- Merge changes
- Restore deleted paragraphs
- See who changed a chapter and when



What Can You Do With Git?

➔ Initialize a project for version control (git init)
➔ Track changes to files (git add, git commit)
➔ View history of your code (git log)
➔ Work on features separately (git branch, git checkout)
➔ Collaborate with others using Git hosting platforms like GitHub, GitLab, or Bitbucket

Signing Up on GitHub

- Go to github.com
- Click Sign Up
- Enter email, create a password, choose a username
- Verify the account via email
- Choose the free plan

Resource: https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github

Installing and Configuring Git Locally

Check if Git is already installed: git --version

If it returns a version number, it's already installed. If not:

- Mac: `brew install git` or download from https://git-scm.com
- Windows: Download Git for Windows from https://git-scm.com/download/win (installs Git Bash too)
- Linux (Ubuntu): `sudo apt install git`

Configure Git with your identity (this is what shows on every commit you make):

- git config --global user.name "Your Name"
- git config --global user.email "you@example.com"

Verify it worked:

- git config --list

Resource: https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

Connecting GitHub to Your Local Machine

Option A: Personal Access Token (PAT)

GitHub no longer accepts passwords over HTTPS. They use a token instead.

Steps to generate a PAT:

1. On GitHub, click profile photo → Settings
2. Scroll to Developer settings (bottom left)
3. Click Personal access tokens → Tokens (classic)
4. Click Generate new token (classic)
5. Add a note (e.g. "my laptop"), set expiration, check repo scope
6. Click Generate token
7. Copy it immediately — GitHub will never show it again

When Git asks for a password in the terminal, paste the token instead.

Resource: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

Option B: SSH Key (Recommended for regular use)

SSH lets you connect without typing a token every time.

Step 1, Generate an SSH key: ssh-keygen -t ed25519 -C "you@example.com"

Press Enter to accept the default file location. Set a passphrase or leave it empty.

This creates two files:

- `~/.ssh/id_ed25519` — the private key (never share this)
- `~/.ssh/id_ed25519.pub` — the public key (this goes to GitHub)

Step 2, Copy the public key by copying the output of this command: cat ~/.ssh/id_ed25519.pub

Step 3, Add it to GitHub:

- GitHub → Settings → SSH and GPG keys
- Click New SSH key
- Give it a title (e.g. "my laptop"), paste the key, save

Step 4, Test the connection: ssh -T git@github.com

If it says "Hi [username]! You've successfully authenticated" — you're connected.

Resource: https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agen

Core Git Workflow

Create a repository on GitHub:

1. Click "+" → New repository
2. Name it, add a README, set to Public, click Create

Clone it locally:

- git clone git@github.com:username/repo-name.git
- cd repo-name

The three-step commit loop (repeat this until it's muscle memory):

Each step:

- `git add .` — stages all changed files (tells Git "include these in the next snapshot")
- `git commit -m ""` — saves the snapshot with a message
- `git push` — sends it up to GitHub

Other commands you'll use constantly:

```
git status      # what's changed?

git log         # history of commits

git pull        # get latest changes from GitHub
```

Resource: https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository

Git Distribution & Commands

1. Staging (git add): Think of this like a pre-check area. Staging basically means you are prepping your work for a commit. You're telling Git: "I want to save these files in the next snapshot."

- git add file.txt # Stage one particular file
- git add . # Stage all changes for all files and folders

2. Committing (git commit): This is when you officially save a snapshot of your code, like hitting "Save" on a big Word doc with a note. Commit = "Save point" + a message

- git commit -m "Added user login feature"
- git commit -m "ch(pipeline): Updated docker image". "ft(frontend): Everything HTML", "fx(bug): Sign in done"

3. Branches (git branch, git checkout): A branch is like a parallel universe of your code. You can add features or test changes without touching the main codebase. Work in isolation, merge when ready.

- git branch feature/set-up-pipeline # Create a branch
- git checkout feature/set-up-pipeline # Switch to it

4. Pushing (git push): You've saved your work locally, now you want to send it to the cloud (e.g. GitHub). Push = Upload changes to the shared repo (remote)

- git push origin main
- git push origin feature/set-up-pipeline

5. Pulling (git pull)

You're about to start coding, but someone on your team pushed changes.

Pull = Download updates from teammates. Run this to get the latest updates from the cloud.

git pull origin main

git pull origin (whatever branch you want to pull from)

6. Remotes (git remote)

This is your link to GitHub, GitLab, etc., the central repo you collaborate through.

Remote = your project's "home" online

git remote -v # View connected remotes

git remote add origin https://github.com/user/repo.git

Git clone: Creates original copy
Git pull: To update an existing codebase
Git fork: Create an independent copy

Git Terms

Repository (Repo): A repository is the project folder that Git is tracking. It contains your code, a hidden .git/ folder, and the entire history of changes. Think of it as your code's home base, whether local or online.

Commit: A commit is a snapshot of your project at a point in time. Each commit has a unique ID and message explaining what changed. It's like hitting "Save" with a note for your team (or future self).

Branch: A branch is a separate line of development. You can create a branch to work on a feature or fix without touching the main code. It's like working on a new chapter of a book without changing the published version.

Merge Conflict: This occurs when two branches changed the same part of a file, and Git doesn't know which change to keep. You'll have to manually resolve it. Think of two teammates editing the same sentence differently, you have to choose or combine their edits.

Working Directory vs. Staging Area: Working dir = your desk. Your actual files and changes (what you're editing right now). Staging area = the "Ready" tray before filing. It is where you put changes you're ready to commit

- git add file.txt # Moves file from working dir to staging area
- git commit # Saves what's in staging to history

HEAD: HEAD is a pointer to the current commit or branch you're on. Think of HEAD as "You are here" on the Git map.

git checkout main # HEAD now points to being on 'main'

Remote: A remote is a link to a copy of your repo hosted elsewhere, like on: GitHub, GitLab, Bitbucket. Local repo = your computer while Remote repo = shared cloud version

git remote add origin https://github.com/you/project.git

git push origin main



Some Common Git Commands



git init	Start tracking a new project
git commit -m "message"	Save your changes with a message
git status	See what's changed or pending
git log	View history of commits
git branch	List or create branches
git merge	Combine changes from one branch into another
git push / git pull	Send changes to or fetch from GitHub or remote repo
git add . (try git add file.txt)	Stage all changes

Some ways to access Git as a tool

1. **Command Line (Terminal or Git Bash):** The most powerful and flexible way to use Git. This is how pros work, and it's scriptable. Great for: Developers, DevOps, scripting, automation.
2. **Git GUI Clients (Graphical Tools):** Visual apps for managing Git without the terminal. Great for: Beginners, visual learners, quick commits.



Examples:

- GitHub Desktop (beginner-friendly)
- GitKraken
- Sourcetree
- Fork
- Tower

3. Code Editors with Git Built-In: Most modern editors let you use Git without typing commands. Great for developers who want Git + code in one place

- VS Code – built-in Git sidebar
- JetBrains IDEs (e.g., IntelliJ, PyCharm) – version control panel
- Sublime Text with plugins

4. Web Interface (GitHub, GitLab, Bitbucket): You can view, edit, commit, fork, and even resolve pull requests directly in the browser. Great for: Quick changes, reviews, or managing repos from any device

5. Mobile Apps: Apps like Working Copy (iOS) or Pocket Git (Android) let you manage code on the go.

HELP with Git?: Git Docs, Man Pages, Google

Practice Questions....



1. What is the difference between `git pull` and `git fetch`?
2. You and a colleague edited the same line in a file. What happens when you merge, and how do you resolve it?
3. What does `git init` do?
4. Why would you use a branch instead of committing directly to `main`?
5. Write the commands to stage all changes and commit them with the message "first commit".

End of discussion.

